With the error reporting level set to `E_ERROR` only, the warning that would have been emitted from `print $anotherUndefVar` is quietly ignored. You achieve the same effect with `ini_set('error_reporting', E_ERROR)`. PHP also offers an operator for controlling error reporting: the `@` symbol. This can be prepended to any expression to suppress any errors that might be generated for that statement:

```
print @$undefVar;
print "All is well?!";

Output:
All is well?!
```

Obviously, you should tread very carefully around code that makes use of the `@` error control operator and minimize its use in your own code. One potential (ab)use of this feature in older PHP code is `$val = @$arr['undef'];` (to suppress warnings if array keys are undefined), which is no longer necessary with the null coalesce operator. `$val = $arr['undef'] ?? null;` is a far cleaner piece of code.

In addition to displaying errors in program output (either in command line output for CLI SAPI programs or as part of the response body content for web server SAPIs), errors can be redirected to `STDOUT` or `STDERR`, logged to a file or the *nix `syslog` subsystem, or even wrapped in HTML tags to make them browser friendly. A common strategy for production PHP systems is to disable errors in program output with `display_errors = Off` and enable an error log file with something like `error_log = php_errors.log`. This makes sense because you almost never want to expose any information about your systems to your users. If `error_log` is not defined, then PHP will default to error logging according to the SAPI – `STDERR` for CLI, or the web server's error log (which means PHP's messages will be mingled with the server's own errors).

Some of the configuration directives are detailed below: